

You have been supplied with the following source files:

**TheData.txt**  
**Blue.txt**  
**Green.txt**  
**Orange.txt**  
**Pink.txt**  
**Red.txt**  
**Yellow.txt**

Open the evidence document, **evidence.doc**

Make sure that your name, centre number and candidate number will appear on every page of this document. This document must contain your answers to each question.

Save this evidence document in your work area as:

**evidence\_** followed by your centre number\_candidate number, for example: **evidence\_zz999\_9999**

A class declaration can be used to declare a record. If the programming language used does not support arrays, a list can be used instead.

Seven source files are used to answer **Question 2**. The files are called **TheData.txt**, **Blue.txt**, **Green.txt**, **Orange.txt**, **Pink.txt**, **Red.txt** and **Yellow.txt**

**1** A program stores positive integers in a circular queue.

The queue is stored as a global 1D array of 20 integers with the identifier `Queue`. Each index is initialised with the data `-1`

The global variable `HeadPointer`, initialised to `-1`, points to the first element in the queue.

The global variable `TailPointer`, initialised to `-1`, points to the last element in the queue.

The global variable `NumberItems`, initialised to `0`, stores the number of items in the queue.

**(a)** Write program code to declare and initialise `Queue`, `HeadPointer`, `TailPointer` and `NumberItems`

Save your program as **Question1\_J25**.

Copy and paste the program code into part **1(a)** in the evidence document.

[2]

**(b)** The function `Enqueue()`:

- takes an integer as a parameter
- checks if the queue is full
- returns `FALSE` if the queue is full
- stores the parameter in the next position in the queue and returns `TRUE` if the queue is **not** full
- updates the appropriate pointers and `NumberItems`

Write program code for `Enqueue()`

Save your program.

Copy and paste the program code into part **1(b)** in the evidence document.

[6]